
Problem 1(a).

```
% Jacobi method

A = [ 10,  -1,  2,  0;
      -1, 11, -1,  3;
        2, -1, 10, -1;
        0,  3, -1,  8];

b = [6; 25; -11; 15];

maxiterations = 100;
tolerance = 10^-4;
x0 = [0; 0; 0; 0];

fprintf('\n -----Solution of Problem 1-----\n\n');

%----- Jacobi
[iteration_J, x_J] = Jacobi(A, b, maxiterations, tolerance, x0);

fprintf('Jacobi Method:\n');
fprintf('Number of iterations: %d\n', iteration_J);
fprintf('Computed solution:\n');
disp(x_J);

%----- Gauss-Seidel
[iteration_S, x_S] = GaussSeidel(A, b, maxiterations, tolerance, x0);

fprintf('\nGauss-Seidel Method:\n');
fprintf('Number of iterations: %d\n', iteration_S);
fprintf('Computed solution:\n');
disp(x_S);

% Problem 1(b): Spectral radius

[M_J, M_GS] = J_GS_itr_mat(A);

rho_J = max(abs(eig(M_J)));
rho_GS = max(abs(eig(M_GS)));

fprintf('\nSpectral radius of Jacobi iteration matrix: %.6f\n', rho_J);
fprintf('Spectral radius of Gauss-Seidel iteration matrix: %.6f\n', rho_GS);

-----Solution of Problem 1-----

Jacobi Method:
Number of iterations: 11
Computed solution:
    0.9999
    2.0001
```

-1.0001
1.0001

Gauss-Seidel Method:
Number of iterations: 5
Computed solution:
1.0001
2.0000
-1.0000
1.0000

Spectral radius of Jacobi iteration matrix: 0.426437
Spectral radius of Gauss-Seidel iteration matrix: 0.089823

Problem 2

```
% Coefficient Matrix
A = [ 4, -1, 0, 0, 0, 0;
      -1, 4, -1, 0, 0, 0;
      0, -1, 4, 0, 0, 0;
      0, 0, 0, 4, -1, 0;
      0, 0, 0, -1, 4, -1;
      0, 0, 0, 0, -1, 4];

% Load vector
b = [0; 5; 0; 6; -2; 6];

maxiterations = 100;
tolerance = 10^-8;
x0 = zeros(6,1);

fprintf('\n -----Solution of Problem 2-----\n\n');

%----- Jacobi
[iteration_J, x_J] = Jacobi(A, b, maxiterations, tolerance, x0);

fprintf('Jacobi Method:\n');
fprintf('Number of iterations: %d\n', iteration_J);
% fprintf('Computed solution:\n');
% disp(x_J);

%----- Gauss-Seidel
[iteration_S, x_S] = GaussSeidel(A, b, maxiterations, tolerance, x0);

fprintf('\nGauss-Seidel Method:\n');
fprintf('Number of iterations: %d\n', iteration_S);
% fprintf('Computed solution:\n');
% disp(x_S);
```

```

%----- SOR

Omega = [1.01, 1.05, 1.1, 1.15];

x_SOR = zeros(size(A,1), length(Omega));
Itr    = zeros(1, length(Omega));

for i = 1:length(Omega)

    [iteration, x] = SOR(A, b, Omega(i), maxiterations, tolerance, x0);

    x_SOR(:,i) = x;
    Itr(i) = iteration;

end

fprintf('\nSOR Method Results:\n\n');

for i = 1:length(Omega)

    fprintf('omega = %1.4f:\n', Omega(i));
    fprintf('Number of iterations = %d\n', Itr(i));
    %fprintf('Computed solution:\n');
    %disp(x_SOR(:,i));

end

%----- Optimal omega

[M_J, ~] = J_GS_itr_mat(A);

rho_J = max(abs(eig(M_J)));

omega_opt = 2/(1 + sqrt(1 - rho_J^2));

[iteration_opt, x_opt] = SOR(A, b, omega_opt, maxiterations, tolerance, x0);

fprintf('\nOptimal omega = %1.4f:\n', omega_opt);
fprintf('Number of iterations = %d\n', iteration_opt);
% fprintf('Computed solution:\n');
% disp(x_opt);

fprintf(['Observation: The Jacobi method required the largest number of
iterations, showing that it converges slowly.\n', ...
        'The Gauss-Seidel method required fewer iterations because it uses
the updated values during the iteration.\n', ...
        'The SOR method required the fewest iterations when the relaxation
parameter omega was chosen optimally.\n']);

%-----Functions-----%
% function for Gauss-Jacobi method
function [iteration, x] = Jacobi(A, b, maxiterations, tolerance, x0)

```

```

%places diagonal elements of A on main diagonal of matrix N
N = diag(diag(A));

%creates matrix P subtracting off main diagonal elements
P = N-A;

%initializes iteration count
iteration = 1;

%produces new approximation x from old approximation x0
x = N\(P*x0+b);

%stops iterative procedure if the relative difference between x and
%x0 (calculated using the l-infinity norm) is satisfactory
while iteration <= maxiterations && norm(b - A*x, 2) / norm(b, 2) >=
tolerance
    %increments the iteration count
    iteration = iteration+1;

    %updates the old approximation to be the new approximation
    x0 = x;

    %produces new approximation x from old approximation x0
    x = N\(P*x0+b);
end
end

% function for Gauss-Seidel method

function [iteration, x] = GaussSeidel(A, b, maxiterations, tolerance, x0)
    %places diagonal and lower triangular elements of A into matrix N
    N = tril(A);

    %creates matrix P subtracting off N from A
    P = N - A;

    %initializes iteration count
    iteration = 1;

    %produces new approximation x from old approximation x0
    x = N\(P*x0 + b);

    %stops iterative procedure if the relative difference between x and
    %x0 (calculated using the l-infinity norm) is satisfactory
    while iteration <= maxiterations && norm(b - A*x, 2) / norm(b, 2) >=
tolerance
        %increments the iteration count
        iteration = iteration + 1;

        %updates the old approximation to be the new approximation
        x0 = x;

        %produces new approximation x from old approximation x0
        x = N\(P*x0 + b);

```

```

    end
end

% function for SOR method

function [iteration, x] = SOR(A, b, omega, maxiterations, tolerance, x0)
    %places diagonal elements of A on main diagonal of matrix D
    D = diag(diag(A));

    %places strictly lower triangular part of A into matrix L
    L = tril(A, -1);

    %places strictly upper triangular part of A into matrix U
    U = triu(A, 1);

    %constructs matrix N for SOR iteration
    N = D + omega*L;

    %constructs matrix P for SOR iteration
    P = (1-omega)*D - omega*U;

    %initializes iteration count
    iteration = 1;

    %produces new approximation x from old approximation x0
    x = N\(P*x0 + omega*b);

    %stops iterative procedure if the relative difference between x and
    %x0 (calculated using the l-infinity norm) is satisfactory
    while iteration <= maxiterations && norm(b - A*x, 2) / norm(b, 2) >=
tolerance
        %increments the iteration count
        iteration = iteration + 1;

        %updates the old approximation to be the new approximation
        x0 = x;

        %produces new approximation x from old approximation x0
        x = N\(P*x0 + omega*b);
    end
end

% Iteration matrices
function [M_J, M_GS] = J_GS_itr_mat(A)
    D = diag(diag(A));
    L = tril(A, -1);
    U = triu(A, 1);
    M_J = -D\(L+U);
    M_GS = -(D+L)\U;
end
%-----%
```

-----Solution of Problem 2-----

Jacobi Method:

Number of iterations: 18

Gauss-Seidel Method:

Number of iterations: 10

SOR Method Results:

omega = 1.0100:

Number of iterations = 9

omega = 1.0500:

Number of iterations = 7

omega = 1.1000:

Number of iterations = 9

omega = 1.1500:

Number of iterations = 11

Optimal omega = 1.0334:

Number of iterations = 7

Observation: The Jacobi method required the largest number of iterations, showing that it converges slowly.

The Gauss-Seidel method required fewer iterations because it uses the updated values during the iteration.

The SOR method required the fewest iterations when the relaxation parameter omega was chosen optimally.

Published with MATLAB® R2024b